

erative AI of Large Language Model, are not about integrating GenAI into workflows. For example, the first example highlights the value of an LLM-based classifier that goes beyond simple keyword search. Although the posting is from a GenAI company, it describes a *customer support* role unrelated to integrating AI into workflows. The model correctly classifies it as a non-integrator position. See Appendix A.13 for additional examples.

Role: Customer Service Representative (HireQuotient)

Summary: HireQuotient is a pioneering company in the Software Development industry, transforming recruitment processes through Generative AI and Skill Intelligence. The position is a Mid-Level customer service representative.... focused on ensuring high-quality support and satisfaction for customers ...

Responsibilities:

- Manage customer inquiries, complaints, and feedback through various channels, ensuring a high level of satisfaction.
- Provide proactive support through live chat, email, and phone.
- Remain informed about product updates and company policies to deliver accurate information.

...

Role: Senior Security Engineer (Offensive Security, BytePlus)

Summary: The team builds infrastructures, platforms, and technologies to protect users, products, and systems. You will contribute to key security initiatives, developing scalable and secure-by-design solutions.

Responsibilities:

- Responsible for risk discovery and penetration testing of cloud products and infrastructure.
- Conduct risk analysis and threat modeling; provide systematic solutions to business lines.
- Research cutting-edge technologies including cloud-native, microservices, zero trust, big data, and large language models.
- Support the development of secure business technologies and architectures.

...

A.14 Task Extraction Pipeline

This appendix documents the pipeline that maps each of the 355,013 raw job-postings in our 1,000-firm sample to a subset of the 19,265 standardized O*NET tasks, producing the 2,748,186 (posting, task) pairs used in Section 5.1.1.

The pipeline has two stages. Stage 1 is a classical information-retrieval step that, given a posting’s raw text, narrows the 19,265-task universe to the $K = 50$ O*NET tasks most textually similar to the posting. Stage 2 is a large language model (LLM) that reads the posting together with these 50 candidates and selects the subset of candidates it judges to be genuinely required by the posting, tagging each selection with a confidence label. The retrieval stage is deliberately occupation-agnostic—we do *not* restrict candidates to tasks listed in O*NET under the posting’s Revelio-assigned 6-digit SOC code—because Revelio’s SOC assignment is itself imperfect and many postings span tasks from adjacent occupations. Stage 2 is responsible for enforcing relevance and is the sole source of the final match decision.

Stage 1: Multi-view TF-IDF retrieval Rather than hashing the posting into a single bag of words, the retriever scores each of the 19,265 O*NET task strings against three views of the posting and combines the three similarities into a weighted score. The three views are:

1. the **full raw text** of the posting, after stripping boilerplate (equal-employment-opportunity statements, benefits enumerations, legal disclaimers);
2. the **duties block**, extracted by locating section headers such as “Responsibilities,” “What you’ll do,” “Key tasks,” or “Duties,” and taking the following paragraph(s) up to the next section header;
3. the **requirements block**, extracted analogously from headers such as “Qualifications,” “Requirements,” or “What we’re looking for.”

When a duties block is detected, the three views are combined with weights (0.40, 0.45, 0.15); otherwise the full-text view carries 0.85 and the requirements view 0.15. Each view is built from TF-IDF over (1, 2)-grams after lowercasing and stop-word removal; the posting’s job title is boosted by a factor of two in every view. Final candidates are the $K = 50$ O*NET tasks with the highest combined similarity, returned without any SOC restriction.

This retrieval step is deterministic, runs locally on a sparse matrix, and serves purely to bound the LLM’s search space so that the Stage 2 prompt remains short enough for batch-API throughput.

Stage 2: LLM selection with constrained JSON output The LLM is OpenAI’s gpt-4o-mini, queried through the OpenAI Batch API (24-hour asynchronous window; batches of up to 10,000 requests per JSONL file). For each posting we send one request whose content is the posting’s raw text and the Stage-1 candidate list (fifty pairs of task_id and task description). The response must conform to a JSON schema with "strict": True and the following structure:

```

{
  "seniority":          "junior" | "senior",
  "years_experience":   integer | null,
  "likely_occupation": string,
  "extracted_tasks":    [
    { "task_id":        <enum of the 50 candidate IDs>,
      "confidence":     "HIGH" | "MEDIUM" | "LOW" }
  ]
}

```

The `task_id` field is constrained at token-sampling time to one of the fifty candidate identifiers, which prevents the model from hallucinating task IDs outside the retrieved set. In our output, *all* 2,748,260 returned task IDs satisfy this constraint.

The system prompt, reproduced verbatim, instructs the model to perform three classifications (seniority, years of experience, likely occupation) and a task-selection step:

You are an expert labor economist analyzing job postings. For each posting, you must:

1. SENIORITY: Classify as “junior” or “senior” based on Revelio’s 7-tier ordinal scale. Mentally place the posting into one of these 7 levels, then collapse to junior/senior:

L1 Entry Level (intern, trainee, entry-level)	→ JUNIOR
L2 Junior Level (jr., associate analyst, bookkeeper)	→ JUNIOR
L3 Associate Level (senior individual contributor, attorney)	→ SENIOR
L4 Manager Level (manager, superintendent, lead)	→ SENIOR
L5 Director Level (director, head of, VP of a function)	→ SENIOR
L6 Executive Level (managing director, partner, SVP)	→ SENIOR
L7 Senior Executive (CEO, CFO, COO, president)	→ SENIOR

2. YEARS OF EXPERIENCE: Extract the minimum years of experience required as an integer. Return null if not mentioned.

3. LIKELY OCCUPATION: Provide your best guess of the O*NET occupation title.

4. O*NET TASKS: From the candidate task list below, select which tasks are genuinely required by this posting.

- You MUST only return `task_id` values that appear in the candidate list. Do NOT invent or modify task IDs.
- Be SELECTIVE: only include tasks clearly implied by the posting duties/responsibilities.
- Typical range: 5-20 tasks per posting.

- For each task, rate confidence: HIGH (explicitly described), MEDIUM (strongly implied), LOW (plausible but not stated).

Return ONLY valid JSON matching the schema exactly.

Batches are submitted sequentially so that the in-flight token budget stays below the organisation quota. A manifest file records the batch_id, submission time, and completion state of every batch. Failed or partially completed batches are resubmitted; successfully completed batches are skipped on reruns. Stage-1 retrieval artefacts (candidate lists per posting) are cached, so reruns of Stage 2 (e.g., after prompt changes) do not require recomputing TF-IDF.

The pipeline produces two parquet files: a *posting-level* table with one row per posting (job_id, rcid, post_date, onet_code, job_title, LLM-inferred seniority, years_experience, likely_occupation, and n_tasks), and a *task-level* table with one row per extracted (posting, task) pair (job_id, task_id, task_text, confidence, valid_id). Table A.4 reports summary statistics.

Table A.4: Task Extraction Output: Summary Statistics

<i>Posting-level</i>		
Postings processed		355,013
classified junior		158,139
classified senior		196,874
Tasks per posting, mean (s.d.)		7.74 (2.65)
Tasks per posting, median		8
<i>Task-level</i>		
Total (posting, task) pairs		2,748,260
Distinct O*NET tasks appearing	16,342 of 19,265 (84.8%)	
Task IDs within the candidate enum		100.00%
<i>Confidence distribution</i>		
HIGH (explicitly described)		58.4%
MEDIUM (strongly implied)		40.1%
LOW (plausible but not stated)		1.4%

Three features of the output are worth highlighting. First, the LLM respects the JSON schema perfectly: every returned task_id is one of the 50 candidates retrieved in Stage 1, so there are no hallucinated tasks. Second, the output is broad—84.8% of the O*NET task universe appears at least once—but selective within each posting: the median bundle has eight tasks, consistent with the prompt’s “typical range 5–20” instruction. Third, the vast majority of selections (98.5%) are labelled HIGH or MEDIUM confidence; in our main analysis we use all three tiers, but all results are robust to dropping the 1.4% of LOW-confidence matches.

Our pipeline is motivated by the task-extraction step in [Hampole et al. \(2025\)](#), but the two exercises differ in input and target. They extract AI *applications* from résumé text and map each application to the single nearest O*NET task via sentence-embedding cosine similarity, in order to construct a firm-level AI-exposure score. We instead extract the *multi-task bundle* of a job posting—the set of O*NET tasks the posting is asking the worker to perform—because our object of interest is the composition of labour demand and